

In []: #1

```
1. Mutability:
A list is a mutable collection in Python, meaning its elements can be changed
A tuple is an immutable collection, meaning once created, its elements cannot be changed

2. Syntax:
A list is defined using square brackets []
a tuple is defined using parentheses ()

3. Built-in methods:
Lists have more built-in methods (e.g., .append(), .remove(), .pop(), .extend(), .insert(), .clear(), .copy(), .reverse(), .sort())
while tuples have only a few methods (e.g., .count() and .index())

4. Memory Consumption: Lists consume more memory than tuples due to their dynamic nature.
5. Performance: Tuples are generally faster and use less memory than lists.
```

In [5]: #2

```
l1 = [1, 2, 3, 4, 5, 6]
l2 = [6, 5, 4, 3, 2, 1]

for i in range(len(l1)):
    if l1[i] > l2[i]:
        print(f"L1 element value {l1[i]} is greater than L2 element value {l2[i]}")
    elif l1[i] < l2[i]:
        print(f"L1 element value {l1[i]} is less than L2 element value {l2[i]}")
    else:
        print(f"L1 element value {l1[i]} is equal to L2 element value {l2[i]}")
```

```
L1 element value 1 is less than L2 element value 6
L1 element value 2 is less than L2 element value 5
L1 element value 3 is less than L2 element value 4
L1 element value 4 is greater than L2 element value 3
L1 element value 5 is greater than L2 element value 2
L1 element value 6 is greater than L2 element value 1
```

In [12]: #3.

```
def create_square_dict():
    numbers = input("Enter a list of numbers separated by space: ").split()
    numbers = [int(num) for num in numbers]
    square_dict = {num: num ** 2 for num in numbers}
    return square_dict
result = create_square_dict()
print("Resulting dictionary:", result)
```

Resulting dictionary: {2: 4}

In [14]: #4.

```
def Calculator(int1, int2, task):

    task = task.lower()
    if task == 'add':
        return int1 + int2
    elif task == 'sub':
        return int1 - int2
    elif task == 'mul':
        return int1 * int2
```

```

elif task == 'div':
    # Handling division by zero
    if int2 == 0:
        return "Error: Division by zero is not allowed"
    else:
        return int1 / int2
else:
    return "Error: Invalid task"
print(Calculator(2, 5, "ADD")) # Should return 7
print(Calculator(7, 1, "SuB")) # Should return 6
print(Calculator(4, 0, "DIV")) # Should return Error: Division by zero i
print(Calculator(10, 2, "mul")) # Should return 20

```

```

7
6
Error: Division by zero is not allowed
20

```

In [16]: #5.

```

def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False
def print_leap_years_21st_century():
    print("Leap Years of the 21st Century:")
    for year in range(2001, 2101): # 21st century is from 2001 to 2100
        if is_leap_year(year):
            print(year)
print_leap_years_21st_century()

```

```

Leap Years of the 21st Century:
2004
2008
2012
2016
2020
2024
2028
2032
2036
2040
2044
2048
2052
2056
2060
2064
2068
2072
2076
2080
2084
2088
2092
2096

```

In [18]: #6.

```

def print_pattern():

```

```

    for i in range(1, 6): # As there are 5 rows
        for j in range(1, i + 1):
            print(j, end=" ") # Print number with a space, without moving cursor
        print() # Move to the next line after each row
print_pattern()

```

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

In [20]: #7.

```

def convert_to_dict(tuple_list):
    return dict(tuple_list)

input_list = [("name", "Virat"), ("surname", "Kohli"), ("yob", 1986)]

result_dict = convert_to_dict(input_list)

print(result_dict)

```

```
{'name': 'Virat', 'surname': 'Kohli', 'yob': 1986}
```

In [28]: #8.

```

def separate_even_odd_indices(input_list):
    even_index_list = []
    odd_index_list = []
    for index, value in enumerate(input_list):
        if index % 2 == 0:
            even_index_list.append(value)
        else:
            odd_index_list.append(value)
    return even_index_list, odd_index_list

def main():
    user_input = input("Enter a list of numbers separated by spaces: ")
    input_list = [int(x) for x in user_input.split()]
    even_list, odd_list = separate_even_odd_indices(input_list)
    print("Even indexed list:", even_list)
    print("Odd indexed list:", odd_list)

main()

```

```

Even indexed list: [1, 3, 5, 7]
Odd indexed list: [2, 4, 6, 8]

```

In [30]: #9.

```

def longest_word(sentence):
    words = sentence.split()
    longest = words[0]
    for word in words:
        if len(word) > len(longest):
            longest = word
    return longest

sentence = input("Enter a sentence: ")
print("The longest word is:", longest_word(sentence))

```

The longest word is: programming

In [32]: #10.

```
def count_frequency(input_list):  
    frequency_dict = {}  
    for item in input_list:  
        if item in frequency_dict:  
            frequency_dict[item] += 1  
        else:  
            frequency_dict[item] = 1  
    return frequency_dict  
input_list = input("Enter a list of elements separated by spaces: ").split()  
input_list = [int(x) if x.isdigit() else x for x in input_list]  
frequency = count_frequency(input_list)  
print("Frequency of each element:", frequency)
```

Frequency of each element: {1: 1, 2: 2, 3: 1, 4: 3, 5: 2}

In []: